

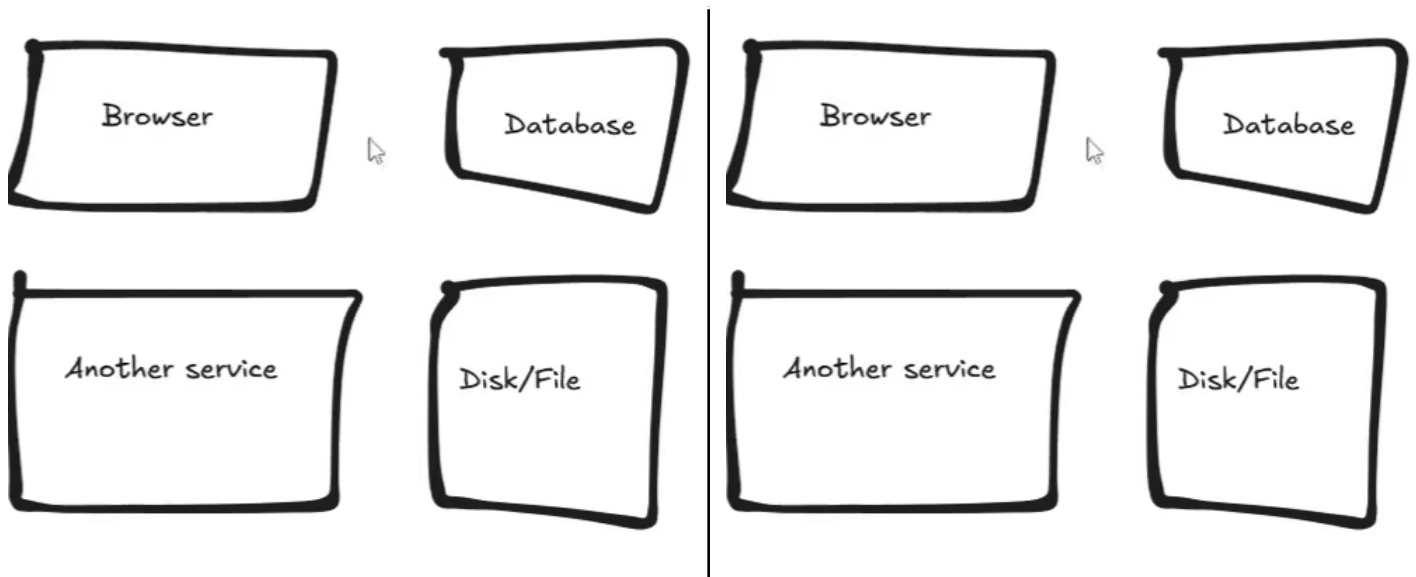
Serialization and Deserialization

The Problem: Data Lives in Memory

All right, we're going to talk about serialization and deserialization. But before we talk about any of that, we need to first discuss the problem that exists regardless of language or framework or text stack. The problem is that your application lives in memory and literally everything else doesn't. When your app is running, the data lives inside of the RAM. objects, struct, classes, whatever language you're using, they only exist while your program is alive.

Memory is temporary
Memory is local
Memory is **not** shared

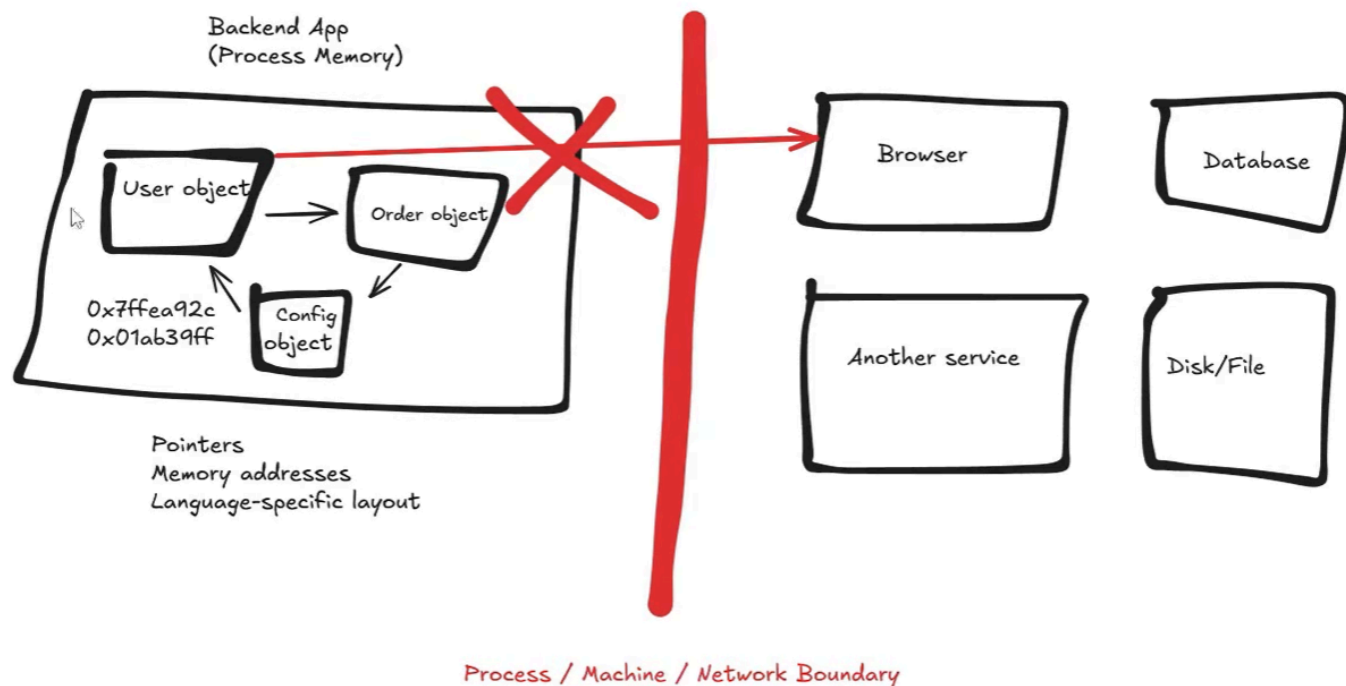
Memory is temporary. It's local and it's not shared. The moment your data needs to leave your process, whether it's going to the browser, another service, a database, or even a file, it stops being your data per se, and it becomes just raw information.



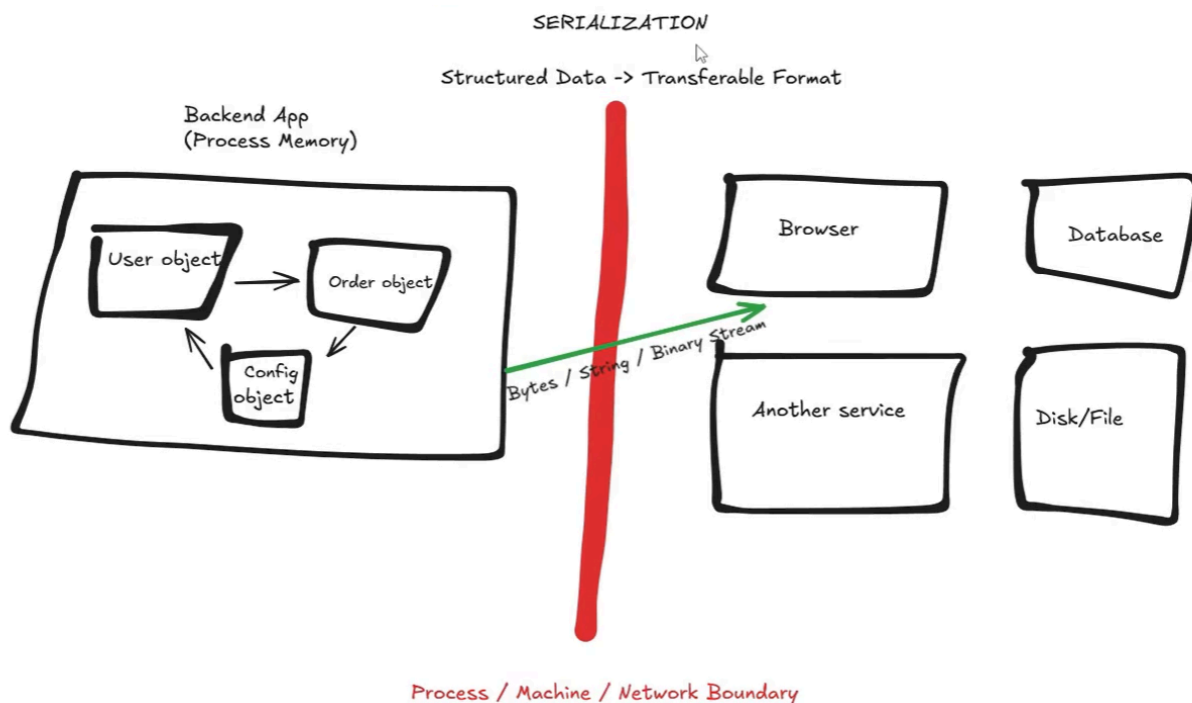
When your back end is running, everything that it knows lives inside memory. And these objects feel real to us, but they only live inside this box.

Browsers, databases, other services, none of these live inside of our process. And this gap right here (| represent gap between two) is where most back-end bugs live.

Data Crossing the Process Boundary



Every time data crosses this line, rules change. You cannot send this(left img) right here(right img). You can't send an object or a class or a pointer to this browser(right img) from inside your process memory(left img). things like pointers, memory addresses, and language specific layout. All of this only makes sense here(left img). Objects are really just patterns of these memory addresses(left img: 0x7ffea92c). These machines over here(right img) don't know what these mean. And even later down the road, the same machine over here(left img) won't know.



So instead of sending the raw memory over here(right img), we send an interpretation of it. something flat and portable and something that can survive outside of the process and be understood outside of the process. This is called **serialization**. And basically what this is is just taking some **structured data** and turning it into a **transferable format**.

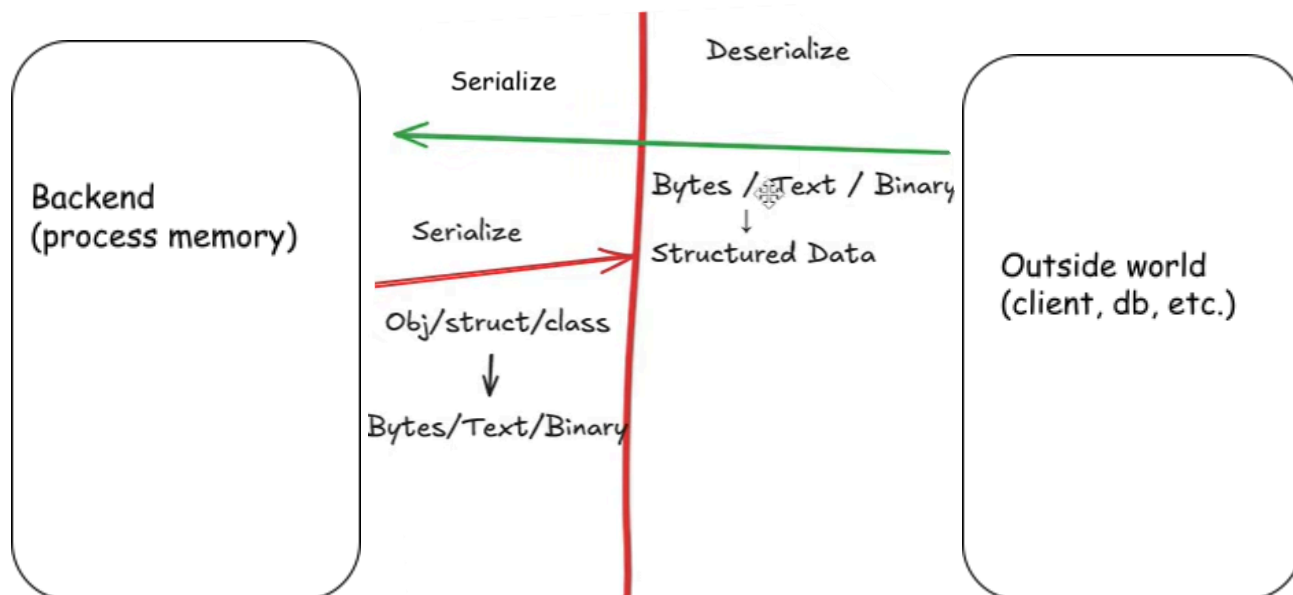
And that transformation from structured data to a transferable format is why serialization exists.

Memory Does Not Cross Boundaries

And something I want you guys to remember is memory does not cross this boundary. But representations do. If you remember nothing else from this video, please remember this. **Memory does not cross boundaries.**

What Are Serialization and Deserialization?

Now that we understand why it exists, let's define it cleanly. These two words can sound scary at first, but they're really not.



Serialization is the process of taking structured data in memory and turning it into something that can cross this boundary right here. Deserialization is simply the inverse of that. take raw data and we rebuild the meaning inside of our program.

Formats vs. Serialization

Formats:

JSON

XML

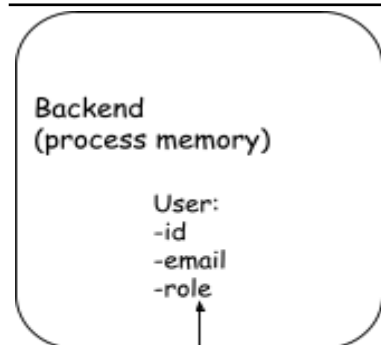
Protobuf

MessagePack

Avro

We have different formats for this.

These are formats and serialization is the concept. Mistake is confusing the two.



Shape changes but the meaning doesn't. The information stays the same. At its core, **serialization is just a translation** and **deserialization is an interpretation**. Same messages, just two different mediums.

Serialization = Translation

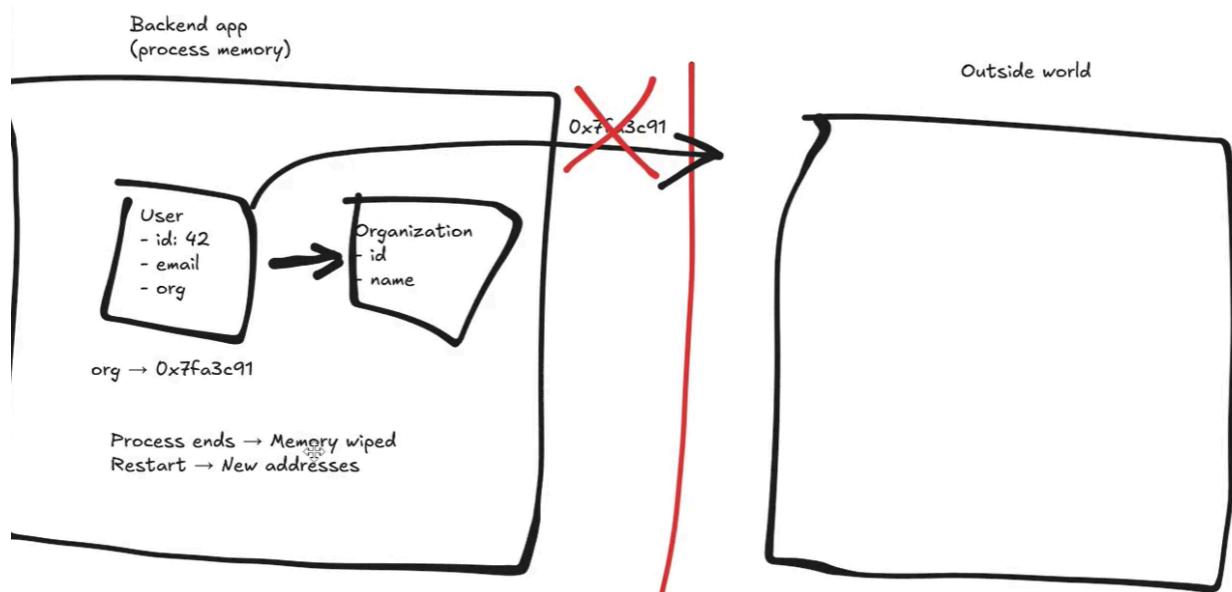
Deserialization = Interpretation

{id: 1, email: abc@gmail.com, role: admin}

Why Can't We Send Objects Directly?

Now, here's where things get interesting. Why can't we just send objects directly? What exactly is inside memory that makes this impossible? To answer that, we need to talk about how memory actually works.

Memory Addresses and Pointers



This looks simple honestly, right? We just think of this (user, org in img) as data. But in memory, this isn't just data. It's a pointer, just a memory address. That address only makes sense here. If I send this pointer to another machine, it's meaningless. Obviously, that memory location doesn't exist in the other machine. or if it does, it has a completely different representation.

When our processes end, memory gets wiped. If we restart, then all of the memory will be allocated to new addresses. Even if you restart the app on the same machine, these addresses change and memory is temporary.

Different Languages, Different Memory

Different languages lay out memory differently. Different runtimes, different garbage collectors.

Even bro, even two Java programs aren't going to agree on memory.

Flat Representation

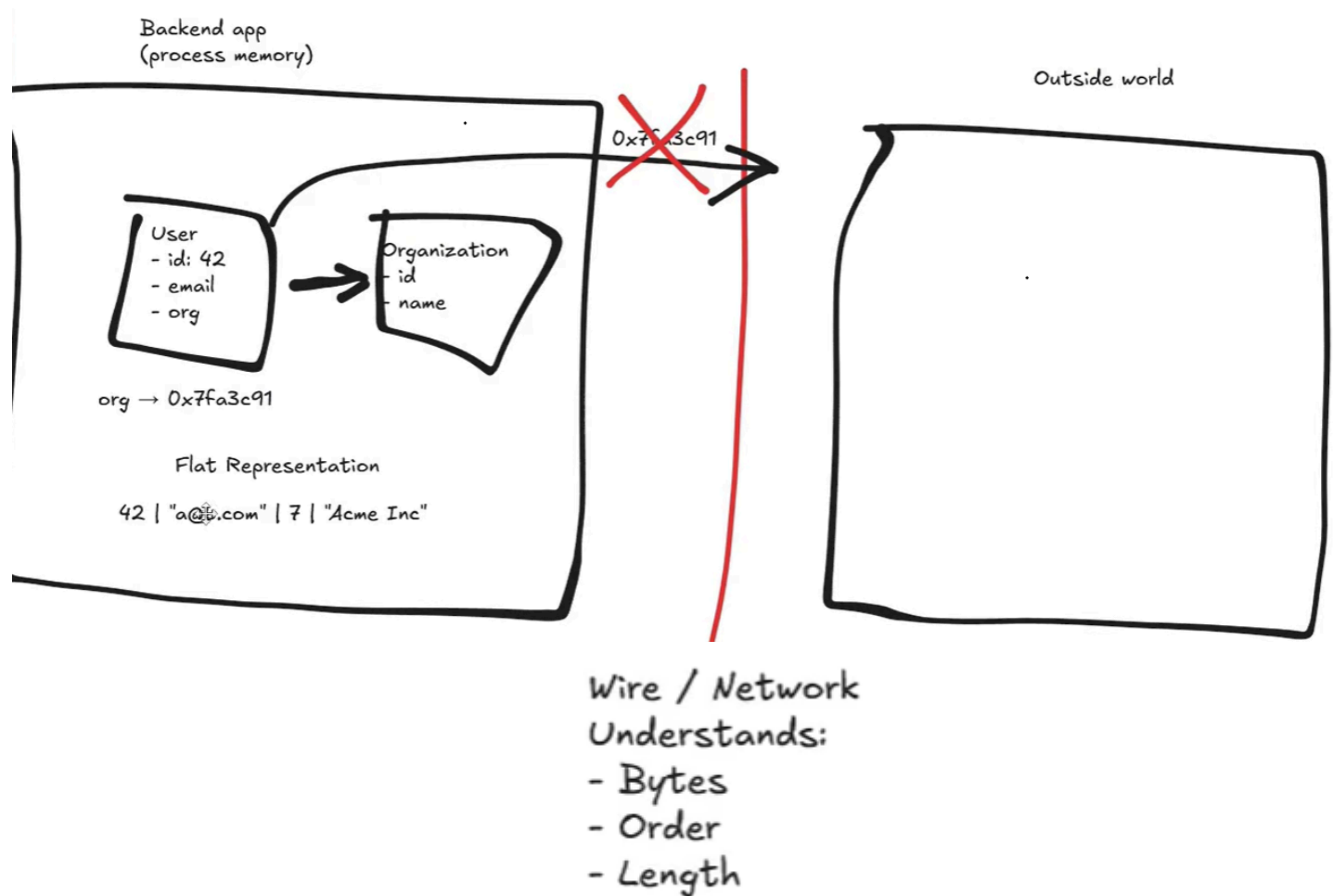
42 | "a@b.com" | 7 | "Acme Inc"

This is just a flat representation of an individual, their user ID, their email, their name, and the organization that they're a part of. So instead of just sending this memory address or this memory directly, we flatten the structure and we extract the values, not the addresses.

Memory Structure $\neq$ Data Meaning

And memory structure does not equal data meaning. And this is super important. Serialization of the data throws away the structure and just keeps the meaning.

Objects in Memory vs. Messages on the Wire



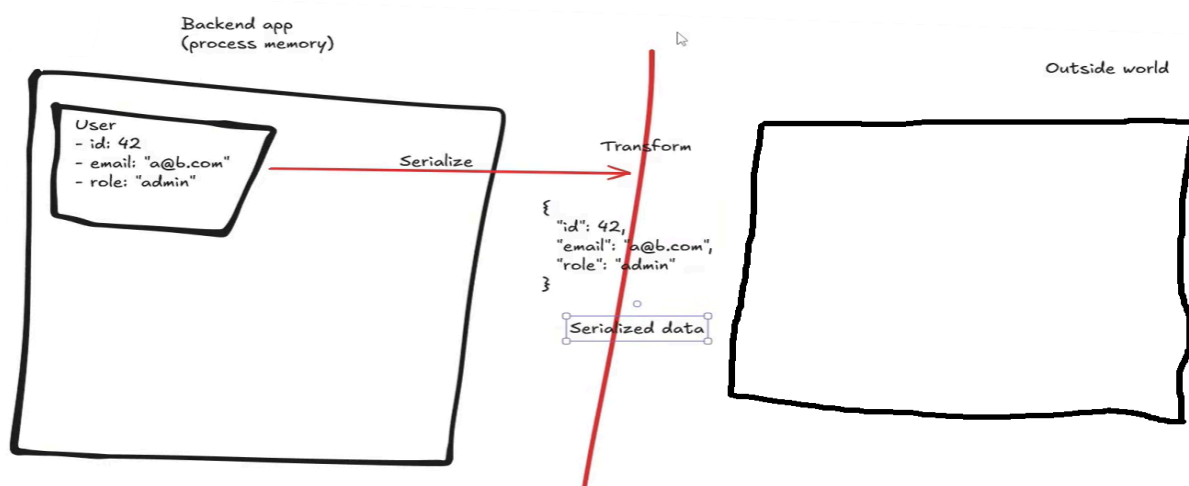
The network doesn't care about your classes. It only understands bytes. The only thing the wire or the network understands is the bytes, the order of them, and the length.

Objects live in memory
Messages live on the wire

The big takeaway here is that objects live in memory, and messages live on the wire.

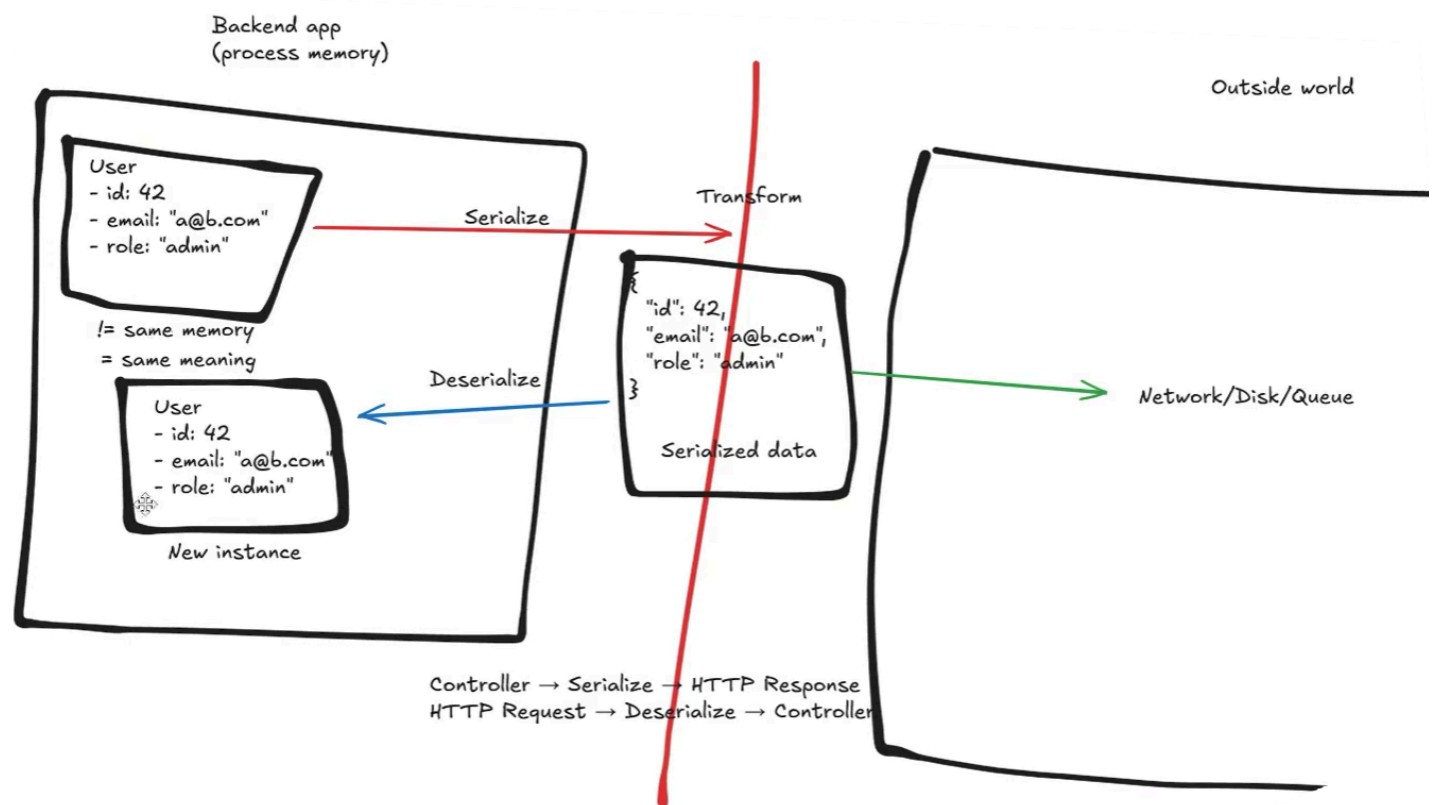
This right here (red line), this empty space up here.

Serialization in Practice



So, let's practice this with a real example. This is normal in-memory data. This is what your backend works with. Before this data can leave the app, it needs to be transformed. And that transformation looks something like this. And this is our serialized data which is transformed. This is no longer an OB object in the memory but is living out here on the wire, but you probably notice this looks familiar if you've seen JSON. Technically, sure, could be an object, but it's just a notation of an object. And this can live on the wire. If you notice, obviously, all of the data is still the same. The form of that data changed.

Data on the Wire



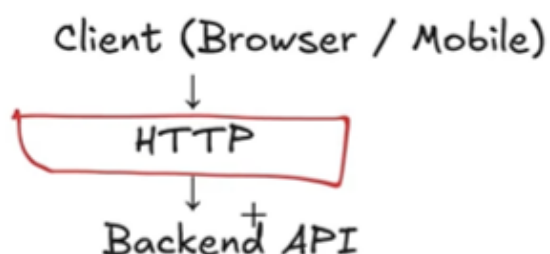
Now this can be sent anywhere to another service, a browser, a database etc. On the receiving side we just reverse the process and we get back the same data but this becomes a new instance and this is not the same object although they look completely similar.

It's a reconstruction and they have the same meaning but it's living in new memory. So not the same memory but same meaning.

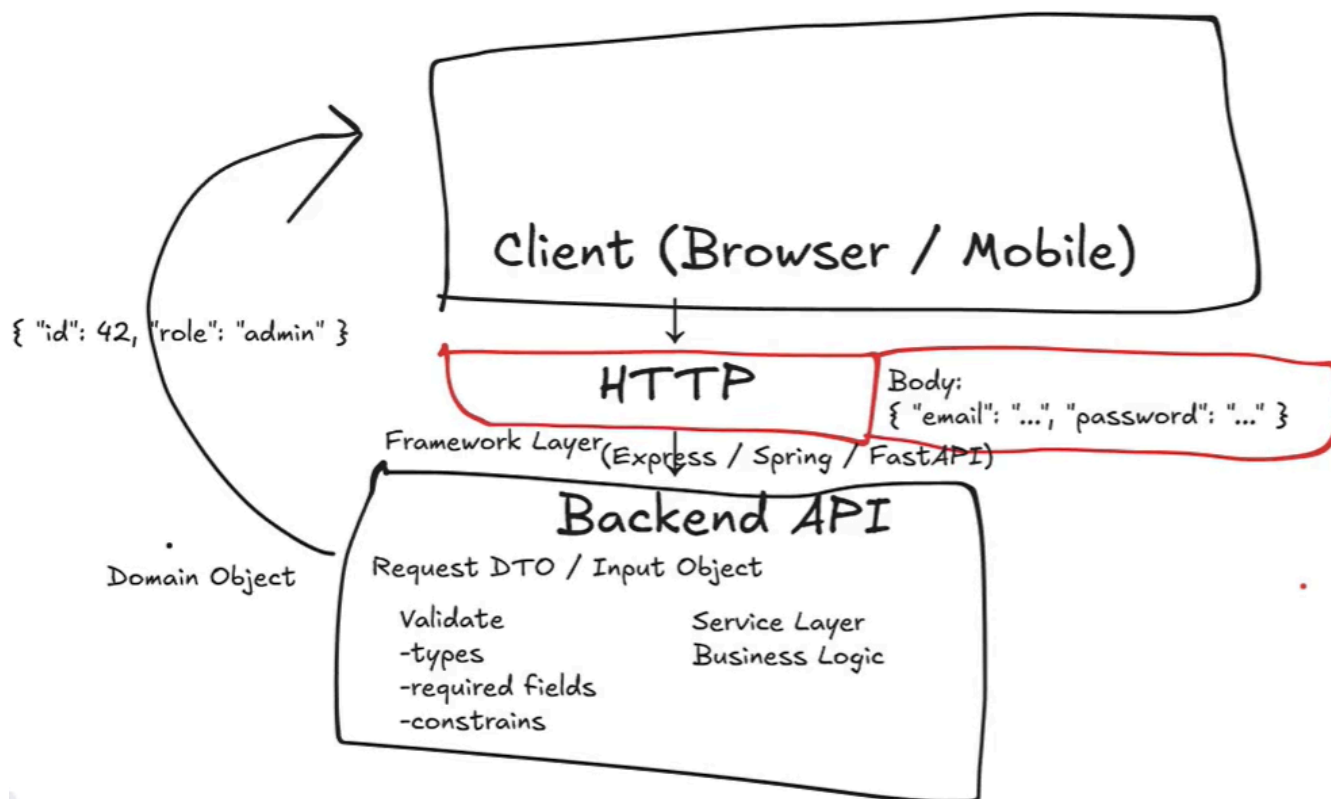
Serialization in an API Call

This is pretty much every API call. A controller serializes the data, sends it to an HTTP response. We send an HTTP request, it deserializes that raw data, and then it sends it back to the controller.

Serialization and Deserialization in APIs



Now that we understand that life cycle, let's look at where this constantly shows up in real backends. Starting with APIs. APIs are simply just serialization pipelines. That's literally it. Our client sends a request and that request gets put into the backend API.



Request Deserialization

So when a request hits your backend, the first thing that happens is deserialization. You might have some sort of framework layer like express, spring, fast API that does it for you, but it's still happening and this input object gets received and a few things happen. We have to validate this, right? Have to figure out certain types, required fields and constraints of the input. But this is where validation lives. Not after but during deserialization.

Business Logic and Structured Data

Business logic never touches unstructured data. It never touches the JSON. It only works with structured data. When you return data, the framework will serialize it.

Send back some sort of domain object and then you'll get this data back.

Content Type

You remember those headers I was talking about in part one? Well, something like *Content-Type: application/json* tells the client exactly how to interpret the bites that it's being given. And no surprise, serialization is a contract.

Request = Deserialize
Response = Serialize

The key mental model here is that the request is deserializing and the response is serializing. And most API bugs honestly are deserialization bugs and not business logic bugs.

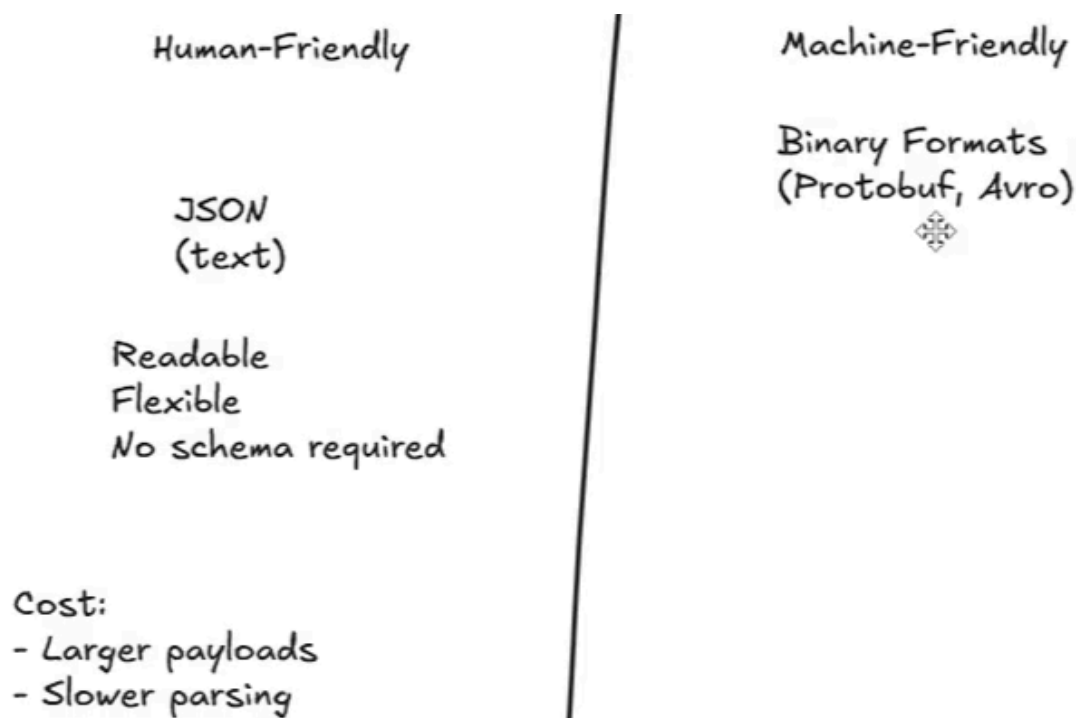
Serialization Formats

So at this point we understand what serialization actually is. But the question now is what format do we actually serialize our data into?

And we have two main categories:

- Human friendly
- Machine friendly

Every serialization format exists to optimize for one of these.



Human-Friendly Formats

JSON exists because obviously humans need to be able to read the data. Things like logs, debugging, curl requests, browser tools, these are all represented in JSON. This means that JSON needs to be readable, flexible, and have no required schema. When I say that JSON has no schema, I just mean that you can add and remove fields with most things continuing to work. And honestly, this is the flexibility that caused REST APIs to explode in popularity. That flexibility isn't free and often times the text is bulky and parsing text is slow compared to binary.

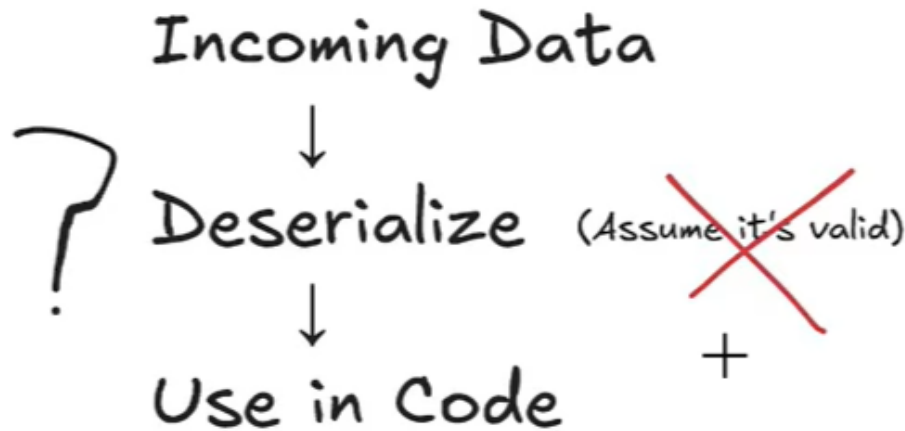
Machine-Friendly Formats

But once humans are no longer in the loop, once systems talk to systems, we stop optimizing for readability naturally.

<u>Format != Serialization</u>	So I want you guys to keep in mind format does not equal serialization.
Public APIs → JSON Internal systems → Binary	This is the rule that most modern real systems follow. It's readable at the edges but inside it's efficient. Public APIs often represent JSON where the internal systems are optimized for binary.

And if performance mattered most, JSON honestly wouldn't exist. But it solely exists so that us humans can read and interpret the data that the systems are working with.

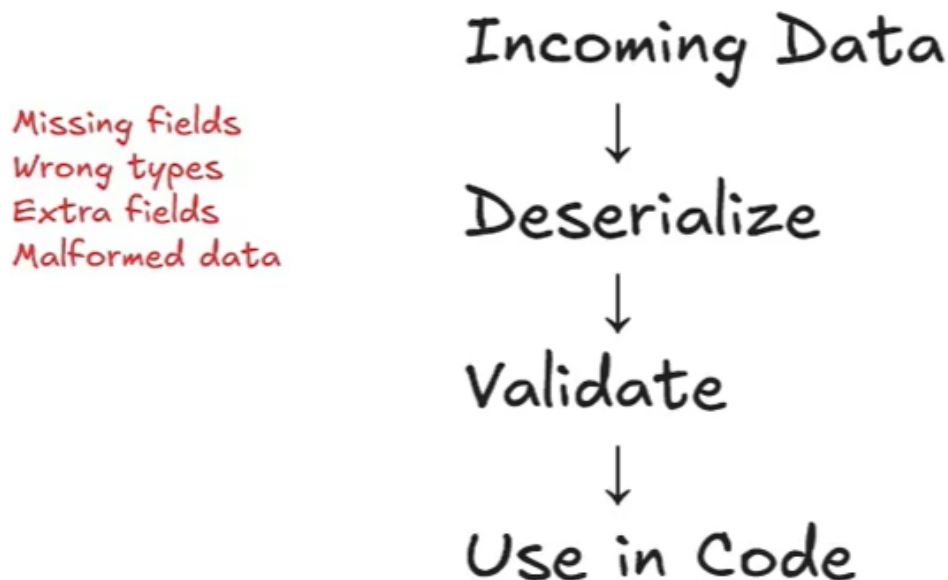
Deserialization and Validation



Now here's the dangerous part. Honestly, serialization is easy, but deserialization is where systems break. Every backend starts here. Raw data entering your system. But assuming that this incoming data is well formatted is a mistake. You never want to assume incoming data is always valid. Valid just meaning more technically that it's wellformed.

Validation During Deserialization

Validation doesn't come after. It happens during deserialization.



Without this validation, we can have things like:

- Missing fields
- Wrong types
- Extra fields
- Malformed data

And honestly, most production bugs are going to live here, not within the business logic.

Untrusted Data

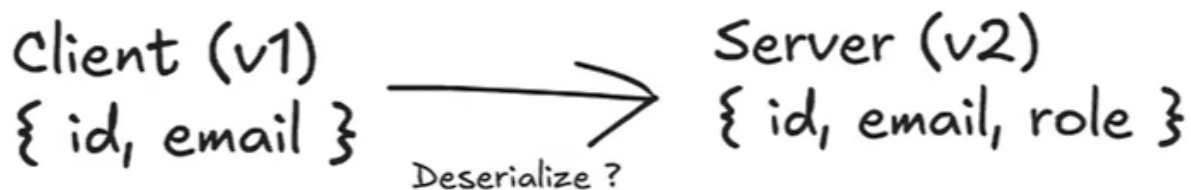
Not all bad data is accidental. Sometimes it's actually even hostile. This is why frameworks are strict. Things like schemas, DTO's, and type checks. These are some of the reasons why backends reject requests. Never, ever, ever trust deserialized data blindly.

Never trust deserialized data. !!!

Once data crosses a boundary of any sort, trust goes back down to zero. This applies to API requests, database reads, message queues, anywhere that your data is being processed.

Serialization and Versioning

Now, we need to talk about one final real world problem. **What happens when data formats change over time?** That's where versioning comes in.



Suppose we have this client server model. Clients and servers often evolve independent of one another asynchronously. So, how do we deserialize this data? This is where things will break if you're careless.

Evolving Schemas Safely

*New fields = optional
Defaults required*

New fields should never break old clients. And a good rule of thumb to follow is whenever you add a new field to your schema, you always want to make them optional and keep the defaults required.

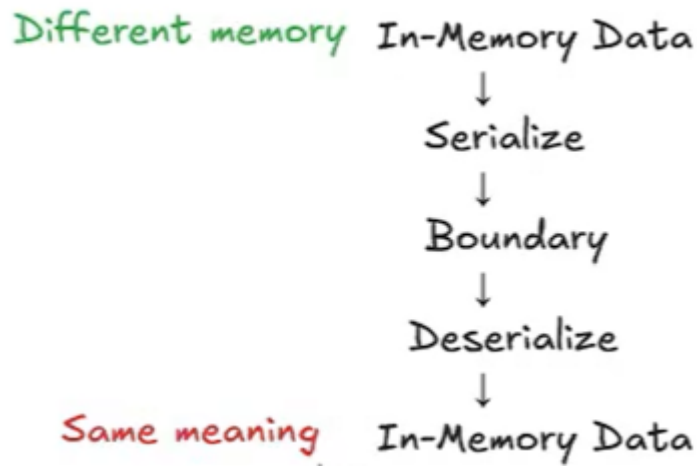
~~Remove fields
Change types~~

Schemas must evolve safely.

However, what you shouldn't do is remove fields or change types. These are breaking changes in your back end. Schemas must be able to evolve safely. And this isn't about format, it's about time.

Final Mental Model

So, at this point, we've seen serialization everywhere. And you're probably sick of hearing the words. So, I'll go ahead and lock in the final mental model.



I want you to understand this right here is the entire story. You have in-memory data. You serialize that data. You send it across a boundary. It is deserialized and then once again becomes in-memory data. This data is different memory across both sides of the pipeline. But it carries the same meaning. Memory can be rebuilt and it can be preserved.

Objects do NOT travel.
Messages do.

And going back to what I said earlier, objects do not travel, but messages do across every boundary across every system. Whether it's an API, a database, cache, Q, file, whatever, this applies all the same.

The One Simple Rule

To break it all down into one simple rule:

If data crosses a boundary,
it gets serialized.

if data crosses a boundary, it gets serialized. So hopefully by the end of this you understand how data crosses between client and server and the outside world and back and forth and how the serialization deserialization process works.